

Engineering Assignment — Creator Communication Platform

Background

You're joining a platform that connects brands with content creators for paid social media campaigns. Brands post jobs, creators apply, and the platform handles content tracking, payments, and payouts. We currently have ~50,000 creators and are growing toward 5,000,000 by end of year.

Communication with creators is the single biggest operational surface of the business. Every creator needs to be told — reliably and promptly — when they've been paid, when their base rate changes, when they've been dropped from a campaign, when a video was approved or rejected, when they need to start warming up, and so on. At the same time, creators constantly ask questions ("when do I get paid?", "why was my video rejected?") and also chat with brands and 8x staff directly.

Today this lives in four disconnected systems, each with its own failure modes, and none of them scale to millions of creators.

Your task: Design a unified Creator Communication Platform that handles automated event-driven messages, inbound FAQ/support via an AI bot, and human↔human chat — all feeding into a single creator inbox.

Current State

Four separate systems glued together:

1. **Human↔human chat** — Supabase Realtime, one conversation per creator↔brand pair. Current implementation subscribes every authenticated client to every message INSERT in the database and filters client-side. Does not scale. Leaks metadata.
2. **Automated system messages** — ad-hoc triggers scattered across the codebase. Payment-sent emails here, drop notifications there, warmup reminders in a cron. No central registry. No idempotency guarantees.
3. **Notifications** — four separate notification systems (brand Slack/Discord, admin Slack, internal email, creator email via Resend). Each has its own config, templates, and retry logic.
4. **Support** — creators message 8x staff with the same ~15 questions every day. Every one is answered by a human. No FAQ, no bot, no knowledge base.

There is no unified inbox. A creator who got paid today, had a video rejected yesterday, got a base-rate change last week, and is in an active chat with their brand sees those in four different places (or not at all).

The Four Surfaces

Your design must address all four and show how they compose:

1. Outbound Automated Messages (event-driven)

Domain events that must reach creators reliably:

Event	Channel priority	Notes
payment.sent	in-app + email	Must never double-send. Must never silently drop.

creator.dropped	in-app + email	Sensitive tone. Often fans out to 100s at once when a campaign ends.
creator.base_rate_changed	in-app + email	Old & new rate, reason, effective date
post.approved	in-app	High volume (thousands/day at scale)
post.rejected	in-app + email	Includes rejection reason from review
warmup.reminder	in-app + push	Time-sensitive (creator's warmup day schedule)
job.offered	in-app + email + SMS	Time-limited offer
viral.alert	in-app	Creator's post went viral — congrats message

Fanout scenarios to handle:

- A campaign ends → 10,000 creator.dropped events fire in the same minute
- A brand bulk-approves 500 posts → 500 post.approved events

2. Inbound FAQ / Support Bot

When a creator sends a message to 8x staff, an AI agent should attempt to answer using:

- Static knowledge base (how payments work, warmup rules, Spark Codes, etc.)
- The creator's own data (their jobs, posts, payment history, warmup status, bank connection state)
- Per-campaign data (brief, content format, platforms, quota)

Examples where the bot should confidently answer:

- "When do I get paid?" → lookup payment schedule + pending balance
- "Why was my video rejected?" → lookup latest post.rejected reason
- "How long is warmup?" → static + creator's current day
- "What platforms do I post on?" → campaign's platform list

Examples where the bot must escalate:

- "I haven't been paid for my last video" (payment disputes)
- "Am I going to be dropped?" (judgment calls)
- Anything the bot is <X% confident about

3. Human ↔ Human Messaging

Creators chat with brands and 8x staff. Realtime. Read receipts. File attachments. Typing indicators. Must scale past a few hundred concurrent users (today's ceiling).

Brands and staff must be able to inject themselves into a thread the bot has been handling — seamlessly, with the full conversation history visible.

4. Unified Inbox

One feed per creator mixing all three surfaces. A creator opening the app sees:

- Chat with their brand (unread: 2)
- System: "You've been paid \$240 for 3 videos" (2h ago)
- Chat with 8x support (bot answered, 1 unread)

- System: "Your base rate changed from \$30 → \$40/video" (yesterday)

Quiet hours. Read state. Deep links into the relevant entity.

Technical Constraints

Constraint	Value
Stack	Next.js (App Router) + PostgreSQL (Supabase) + Vercel
AI model	Claude API (Anthropic) — any model size
Realtime	Supabase Realtime available; Inngest/Queues/cron also on the table
Email/SMS/WhatsApp	Resend (email), Twilio (SMS), WhatsApp Business available
Vercel function timeout	300s default, longer with Fluid Compute
Target scale	50k creators now → 5M by end of year
Burst fanout	10k events in <1 minute must not melt the system
Automated message delivery	At-least-once minimum; exactly-once preferred for <code>payment.sent</code>
Bot response time	< 5s p50, < 10s p99
Bot cost	< \$0.01 per answered message

Your Task

Design the system. We care more about your thinking than the exact implementation.

Deliverables

1. Architecture Overview

- Diagram the end-to-end flow for each of the four surfaces and show how they share infrastructure (inbox table? event bus? notification service?)
- Where does each piece run — Vercel route handler, cron, Inngest/queue worker, Supabase function, client?
- How does a single creator "inbox" materialize? Is it a table, a view, a query-time merge?
- What's the boundary between the messaging system and the rest of the product?

2. Event Pipeline for Automated Messages

- How is an event like `payment.sent` emitted, routed, rendered into a message, and delivered across channels?
- Idempotency — how do you guarantee `payment.sent` never double-sends, even under retries and redeploys?
- Fanout — how do you handle 10k `creator.dropped` events in the same minute without overwhelming email providers, hitting rate limits, or timing out a cron?
- Template system — how are per-event templates defined, localized, and previewed?

3. FAQ / Support Bot

- What triggers the bot vs. the human queue?

- How does it fetch and inject context (creator data + campaign data + knowledge base)?
- Confidence model — what makes the bot escalate?
- How do 8x staff take over a bot-handled thread?
- How do you measure whether the bot is correct without reading every conversation?

4. Human ↔ Human Messaging

- How do you fix the realtime fanout / RLS-leak problem?
- How do you keep it scalable at 100k concurrent creators?
- Read receipts, typing, attachments — in scope or deferred?

5. Unified Inbox

- Data model for the inbox. Is every system message a row in `messages` ? A separate `inbox_items` table? A query-time union?
- How does the client subscribe to updates efficiently?
- Read state, quiet hours, digest rollups (do you batch 50 `post.approved` into one "50 posts approved today" card?).

6. Trade-offs & Failure Modes

- Where did you pick "fast to ship" over "right long-term"? Where did you pick the opposite?
- What breaks at 500k creators? At 5M?
- What's the migration path from the current four-system mess to your design?
- What does this cost per month at 5M creators? Where are the cost cliffs?

7. What You'd Build First

- You have 1 week to ship something that *moves the needle*. What do you cut? What do you keep? What do you defer?

What We're Looking For

This is a big problem. We're evaluating:

- **System judgment** — do you make sound architectural calls under ambiguity, or do you just list options?
- **Tradeoff reasoning** — do you name what you're giving up, or only what you're gaining?
- **Ability to challenge AI** — when Claude/GPT tells you "just use Supabase Realtime for everything" or "use RabbitMQ," do you push back with the actual constraints of this problem?
- **Prioritization** — can you tell the difference between "must solve" and "nice to have"?
- **Clarity** — can you explain a complex design to a CTO in one call?

How You Work

We use AI heavily in our engineering workflow and want to see that you can leverage it effectively.

Using AI tools (Claude, Codex, ChatGPT, Gemini — whatever you prefer) is expected, not penalized.

Include a section describing:

- How you used AI during this assignment — as a thought partner, a reviewer, a code-gen tool?
- Where it helped and where you had to override or correct it
- At least one example where you disagreed with the AI and were right
- 2-3 screenshots of interesting AI interactions
- **How much time you spent on this assignment in total**

Attach the raw transcript(s) of your AI sessions if you can — we genuinely read them.

Evaluation Criteria

Criteria	Weight
System design judgment — do the architectural calls hold up?	25%
Tradeoff reasoning — naming what you sacrifice, not just what you gain	20%
Idempotency / reliability / scale thinking	15%
Bot design — escalation, context injection, measurement	15%
AI usage — quality of thought partnership, pushback, and override	15%
Clarity of communication	10%

Timeline

48 hours from receiving this assignment.

We're evaluating your thinking process and judgment, not your ability to write production code under time pressure. Diagrams, pseudocode, and clear prose beat polished code that doesn't show understanding. If you do write code, keep it to focused snippets that illustrate a specific decision.

Submission

Send your completed assignment as a PDF or Google Doc to theo@8x.social within 48 hours. Once submitted, text me on WhatsApp or LinkedIn to confirm, and we'll schedule a call the following day to walk through it together.